



Coste y presupuesto de contexto (tokens, latencia, calidad)

El contexto es útil... pero no es gratis:

- más contexto ⇒ **más tokens de entrada**
- más tokens ⇒ **más coste** y **más latencia**
- más texto ⇒ más probabilidad de **ruido**

Por eso, en Context Engineering trabajas con un concepto práctico:

Presupuesto de contexto: cuánto puedes permitirte enviar por petición sin romper calidad, coste ni tiempo.

Objetivos

- Entender la relación entre tokens, coste y latencia.
- Medir tokens en desarrollo: `count_tokens` y `usage_metadata`.
- Aplicar estrategias simples de recorte (top-N, filtros, campos mínimos).
- Diseñar un "presupuesto" didáctico para tus proyectos.

1) Qué cuenta como tokens (visión práctica)

En general, en cada llamada pagas (y esperas) por:

- **input tokens:** prompt + historial + datos inyectados
- **output tokens:** lo que genera el modelo
- **tokens extra** (si aplica): "thinking", herramientas, etc.

No necesitas memorizar números exactos ahora. Necesitas:

1. **medir** lo que envías
2. **recortar** cuando se dispare
3. **registrar** para entender el comportamiento real

2) Medir antes de enviar: `count_tokens`

Puedes estimar el tamaño de tu prompt antes de llamar al modelo:

```
from google import genai

MODEL = "gemini-3-flash-preview"
client = genai.Client()

def tokens_de(contenido: str) -> int:
    r = client.models.count_tokens(model=MODEL, contents=contenido)
    return int(r.total_tokens or 0)

prompt = "Hola. Explica en 3 frases qué es un embedding."
print("tokens:", tokens_de(prompt))
```

Esto es útil para poner guardas:

```
MAX_TOKENS_INPUT = 8_000 # umbral didáctico (no es un límite oficial)

def assert_presupuesto(prompt: str) -> None:
    t = tokens_de(prompt)
    if t > MAX_TOKENS_INPUT:
        raise ValueError(f"Prompt demasiado grande: {t} tokens (máx {MAX_TOKENS_INPUT})")
```

3) Medir después de generar: `usage_metadata`

Después de una generación, puedes ver tokens reales consumidos:

```
from google import genai

MODEL = "gemini-3-flash-preview"
client = genai.Client()

response = client.models.generate_content(
    model=MODEL,
    contents="Explica qué es un embedding en 2 frases.",
)

print(response.text)
print("usage:", response.usage_metadata)
```

Campos típicos dentro de `usage_metadata`:

- `prompt_token_count` (entrada)

- `candidates_token_count` (salida)
 - `total_token_count` (total)
 - `thoughts_token_count` (si el modelo usa tokens de pensamiento)
-

4) Medir latencia en Python

El presupuesto no es solo tokens. También es **tiempo**. Medición mínima:

```
import time
from google import genai

MODEL = "gemini-3-flash-preview"
client = genai.Client()

started = time.time()
response = client.models.generate_content(model=MODEL, contents="Resume: ...")
elapsed_ms = int((time.time() - started) * 1000)

print("ms:", elapsed_ms)
print("tokens:", response.usage_metadata.total_token_count)
```

En una app real, este `elapsed_ms` es parte del UX.

5) Estrategias simples para “cortar” contexto

5.1 Top-N / últimos N

Lo más fácil y sorprendentemente útil:

- últimos 10 mensajes
- últimos 3 eventos de historial

5.2 Filtrar antes de inyectar (Python primero)

Si tienes un CSV de 50 000 filas, no lo mandes entero.

Ejemplo: el usuario pide un Código de producto ⇒ filtra la fila con python/Pandas primero ⇒ inyecta solo esa fila en el prompt.

De esta manera, solo inyectamos la fila que necesitamos, evitando inyectar todo el CSV y consumir más tokens de los necesarios.

5.3 Campos mínimos

Si tu perfil de usuario tiene 30 claves, igual solo necesitas 4:

- `language`
- `level`

- `tone`
- `goal`

Considera filtrar los campos que no necesites para ahorrar tokens. Recuerda que podemos usar nuestras habilidades de Python para filtrar los campos que no necesitamos.

5.4 Evitar duplicados

Un problema típico: repetir la misma política, el mismo FAQ o el mismo “perfil” en cada parte del prompt. Si ya lo incluyes, no lo repitas. Si tienes un CSV con 50 000 filas, no lo mandes entero. Dedícate a filtrar los campos que necesitas.

6) Diseñar un presupuesto didáctico (reglas concretas)

Sin números mágicos, pero con reglas claras:

1. **Define un umbral de tokens de entrada** (por ejemplo 8 000).
2. Si lo superas: recorta por prioridad (relevancia), no por azar.
3. Si la latencia sube: reduce output (instrucciones de longitud) o recorta contexto.
4. En modo desarrollo: imprime tokens y tiempo para aprender la relación.

Ejemplo de wrapper:

```
import time
from google import genai

MODEL = "gemini-3-flash-preview"
client = genai.Client()

MAX_TOKENS_INPUT = 8_000

def safe_generate(prompt: str) -> str:
    tc = client.models.count_tokens(model=MODEL, contents=prompt)
    if (tc.total_tokens or 0) > MAX_TOKENS_INPUT:
        raise ValueError(f"Input demasiado grande: {tc.total_tokens} tokens")

    started = time.time()
    r = client.models.generate_content(model=MODEL, contents=prompt)
    elapsed_ms = int((time.time() - started) * 1000)

    um = r.usage_metadata
    print("ms:", elapsed_ms, "| in:", um.prompt_token_count, "| out:",
          um.candidates_token_count, "| total:", um.total_token_count)
    return (r.text or "").strip()
```

Este patrón nos prepara para pensar en “LLM como recurso”, no como magia. Recordemos que todo al final vale dinero y que el coste de los tokens es un factor importante a la hora de diseñar nuestro sistema.

Resumen

- Contexto consume tokens $\Rightarrow$ tokens impactan coste y latencia.
- Mide antes (`count_tokens`) y después (`usage_metadata`).
- Recorta con intención: top-N, filtros, campos mínimos, evitar duplicados.